

FinSight — Bản đặc tả kỹ thuật chi tiết

Implementation Spec theo từng pha

Mỗi quyết định: thư viện, tham số, schema, thuật toán, fallback

Mục đích tài liệu. Đây là spec triển khai — trả lời “*làm như thế nào cụ thể*”, không phải “*làm cái gì*”. Mọi lựa chọn đều kèm tham số mặc định, ngưỡng, và đường fallback. Tài liệu này đi cùng `finsight_ke_hoach.tex` (bản tổng quan kiến trúc).

Contents

1	Quy ước chung	1
2	Pha 1 — OCR giữ cấu trúc (chi tiết)	1
2.1	Phân loại trang: digital vs scan	1
2.2	Pipeline OCR + layout cho trang scan	1
2.3	Thuật toán hậu xử lý ô gộp & phân cấp hàng (BCTC)	2
2.4	Schema đầu ra (document model)	2
2.5	Đánh giá Pha 1	2
3	Pha 2 — Trích xuất & Chuẩn hóa (chi tiết)	2
3.1	Mô hình trích xuất bảng (phần DL lõi)	3
3.1.1	Cấu hình LoRA (khi finetune phương án B — Qwen2-VL-2B)	3
3.1.2	Xây dữ liệu huấn luyện	3
3.1.3	Pipeline inference từng bước	3
3.2	Parse số tiếng Việt	4
3.3	Chuẩn hóa → mã số TT200 (cascade 4 tầng)	4
3.4	Validation số học (bắt lỗi extraction)	4
3.5	Schema chuẩn hóa (1 dòng, lưu Postgres)	5
4	Pha 3 — GraphRAG đa năm (chi tiết)	5
5	Pha 4 — Conformal Prediction (chi tiết)	5
6	Pha 5 — LangGraph Agent (chi tiết)	6
6.1	Graph state schema	6
6.2	Các node & trách nhiệm	6
6.3	Cạnh & điều kiện (conditional edges)	6
6.4	Tools (chữ ký + luật)	6
6.5	Memory store	7
7	Pha 6 — Vòng HITL & Finetune (chi tiết)	7
7.1	Định dạng dữ liệu correction (JSONL)	7
7.2	Trigger huấn luyện lại (drift-based, không theo lịch)	7
7.3	Quy trình finetune	7
7.4	Chiến lược merge & phục vụ	8
7.5	Cổng promote & rollback	8

8	Pha 0/7/8/9 — Hạ tầng, hiệu năng, đánh giá, vận hành (chi tiết ngắn)	8
8.1	Pha 0 — Hạ tầng	8
8.2	Pha 7 — Hiệu năng & song song	8
8.3	Pha 8 — Đánh giá	8
8.4	Pha 9 — Vận hành	9
9	Phụ lục — Bảng tổng hợp ngưỡng & tham số	9

1 Quy ước chung

- **Ngôn ngữ:** Python 3.11; quản lý phụ thuộc bằng uv (hoặc Poetry).
- **Đơn vị tiền chuẩn hóa nội bộ:** VND (đồng), kiểu `int64` hoặc `Decimal` — KHÔNG dùng float cho tiền.
- **Định danh:** `doc_id` (1 file BCTC), `report_id` (1 báo cáo trong file: BS/IS/CF/Notes), `nam` (năm tài chính), `ma_so` (mã số chỉ tiêu TT200).
- **Provenance bắt buộc:** mọi số liệu mang theo (`doc_id`, `page`, `bbox`) để truy vết & trích dẫn.

2 Pha 1 — OCR giữ cấu trúc (chi tiết)

2.1 Phân loại trang: digital vs scan

[QUYẾT ĐỊNH] PyMuPDF (`fitz`). Với mỗi trang, đếm ký tự text trích được. Ngưỡng: `n_chars >= 50` và diện tích text-span phủ $> 5\%$ trang $\Rightarrow$ trang *digital*; ngược lại $\Rightarrow$ *scan*.

- Digital: lấy text + bbox bằng `page.get_text("dict")` (blocks $\rightarrow$ lines $\rightarrow$ spans, có bbox). Bảng thử trước bằng `page.find_tables()`.
- Scan: render ảnh `page.get_pixmap(dpi=300)` $\rightarrow$ đưa sang pipeline OCR.
- Trang lai (vừa text vừa ảnh chèn): xử lý theo vùng — vùng ảnh OCR, vùng text đọc thẳng.

2.2 Pipeline OCR + layout cho trang scan

[QUYẾT ĐỊNH] Bố cục & cấu trúc bảng: **PaddleOCR PP-Structure** (model layout PP-DocLayout; table structure SLANet). Nhận dạng chữ tiếng Việt: **VietOCR** (`vgg_transformer`) chạy trên từng dòng cắt ra — vì recognition đa ngữ của Paddle yếu với dấu tiếng Việt.

Pipeline từng bước:

1. PP-DocLayout $\rightarrow$ phân vùng: {title, text, table, figure, header, footer, page-number}.
2. Với vùng **text**: PP-Structure text-detection $\rightarrow$ box từng dòng $\rightarrow$ crop $\rightarrow$ **VietOCR** nhận dạng.
3. Với vùng **table**: SLANet $\rightarrow$ cấu trúc bảng (HTML + bbox từng ô + row/col span) $\rightarrow$ crop từng ô $\rightarrow$ VietOCR nhận dạng nội dung ô.
4. Thứ tự đọc (reading order): thuật toán **XY-cut** đệ quy; nếu 1 cột thì sort theo **top** rồi **left**.

2.3 Thuật toán hậu xử lý ô gộp & phân cấp hàng (BCTC)

Bảng cân đối có *thực lẽ mang ngữ nghĩa* (Tài sản ngắn hạn > Tiền > Tiền mặt). Thuật toán gán cấp:

1. Lấy tọa độ `x_left` của text trong ô nhân (cột đầu) của mọi hàng.
2. Gom cụm các giá trị `x_left` thành các *mức thực lẽ* (làm tròn về bội số $\approx 12-16\text{px}$, hoặc 1D k-means $k=3..5$).
3. Hàng có mức thực sâu hơn = con của hàng gần nhất phía trên có mức thực nông hơn $\Rightarrow$ dựng cây phân cấp.
4. Ô gộp (row/col span từ SLANet): nhân bản giá trị cho các ô con khi cần, đánh dấu `is_merged`.
5. Gán cột số $\rightarrow$ kỳ: đọc hàng header (“Số cuối năm” / “Số đầu năm”) để map cột $\rightarrow$ `nam`.

2.4 Schema đầu ra (document model)

```
{
  "doc_id": "vnm_2023",
  "pages": [
    {
      "page": 4, "is_scanned": true,
      "blocks": [
        {"type": "title", "text": "BANG CAN DOI KE TOAN", "bbox": [...]},
        {"type": "table", "table_id": "t1",
          "cells": [
            {"row": 3, "col": 0, "text": "Tien va tuong duong tien",
              "indent_level": 1, "row_span": 1, "col_span": 1, "bbox": [...]},
            {"row": 3, "col": 1, "text": "110", "bbox": [...]},
            {"row": 3, "col": 2, "text": "12.345.678.901", "bbox": [...]}
          ],
          "col_header_map": {"2": "so_cuoi_nam", "3": "so_dau_nam"}}
      ]
    }
  ]
}
```

2.5 Đánh giá Pha 1

- Text: **CER/WER** bằng `jiwer`. Mục tiêu CER < 3% trên trang digital, < 8% trên scan.
- Cấu trúc bảng: **TEDS** (Tree-Edit-Distance Similarity, script gốc PubTabNet). Mục tiêu TEDS > 0.90.

3 Pha 2 — Trích xuất & Chuẩn hóa (chi tiết)

3.1 Mô hình trích xuất bảng (phần DL lõi)

[QUYẾT ĐỊNH] Hai phương án, chọn theo dữ liệu sẵn có:

- **A — Decoupled (khuyến dùng cho MVP):** Table Transformer (TATR, DETR-based) cho *structure recognition* + VietOCR cho nội dung ô. Dễ debug, ít dữ liệu, lỗi tách bạch.
- **B — End-to-end:** Qwen2-VL-2B-Instruct (hoặc Donut) finetune ảnh-bảng $\rightarrow$ JSON. Mạnh hơn khi bảng phức tạp, nhưng cần nhiều dữ liệu & khó debug.

MVP: đi phương án A; chuẩn bị dữ liệu để nâng cấp B sau.

3.1.1 Cấu hình LoRA (khi finetune phương án B — Qwen2-VL-2B)

```
LoraConfig(
  r=16, lora_alpha=32, lora_dropout=0.05, bias="none",
  task_type="CAUSAL_LM",
  target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                  "gate_proj", "up_proj", "down_proj"],
  # tùy chọn: thêm lop merger của vision encoder
)
# Train: lr=1e-4, cosine schedule, warmup_ratio=0.03,
#       bf16, grad_accum -> effective batch=16, epochs=3..5,
#       max_grad_norm=1.0. Framework: PEFT + transformers/TRL.
```

Donut (encoder-decoder MBART): LoRA $r=8..16$, `target_modules=["q_proj", "k_proj", "v_proj", "out_proj"]` ở decoder.

3.1.2 Xây dữ liệu huấn luyện

- **Annotation:** PPOCRLabel (xuất cấu trúc bảng HTML, hợp PaddleOCR) hoặc Label Studio (template OCR/table). Nhãn: `bbox ô + text + row/col span + mã số`.
- **Tổng hợp (synthetic) để pretrain:** render hàng nghìn ảnh bảng BCTC từ *chart of accounts TT200* + số ngẫu nhiên hợp lệ, đa font/độ nhiễu/đường kẻ. Ground-truth sinh kèm $\Rightarrow$ miễn phí, vô hạn.
- **Thật:** 15–20 bảng làm golden eval; ~ 50 –200 bảng người sửa (từ HITL) để finetune.
- **Chiến lược:** pretrain trên synthetic $\rightarrow$ finetune trên dữ liệu thật đã sửa.
- **Target format (phương án B):** chuỗi JSON các hàng/ô (xem schema dưới).

3.1.3 Pipeline inference từng bước

1. Ảnh trang $\rightarrow$ phát hiện vùng bảng (PP-DocLayout).
2. Crop bảng $\rightarrow$ TATR (structure) *hoặc* Qwen2-VL (JSON).
3. (Phương án A) Crop từng ô $\rightarrow$ VietOCR $\rightarrow$ text.
4. Lắp lưới ô + áp thuật toán phân cấp/ô gộp (Pha 1).
5. **Parse số** (xem dưới) + phát hiện đơn vị tiền.
6. **Chuẩn hóa nhãn** $\rightarrow$ mã số **TT200** (mục 3.3).
7. **Validation số học** (mục 3.4).

3.2 Parse số tiếng Việt

[QUYẾT ĐỊNH] Regex + luật: dấu chấm là phân tách nghìn (12.345.678); số âm trong ngoặc (1.234) $\rightarrow$ -1234; ô {"-", "", "N/A"} $\rightarrow$ null; nhân hệ số đơn vị ("triệu đồng" $\rightarrow \times 10^6$). Lưu dạng int VND.

3.3 Chuẩn hóa $\rightarrow$ mã số TT200 (cascade 4 tầng)

Insight cốt lõi: BCTC theo TT200 đã in sẵn cột “Mã số” (110, 200, 311...). Vì vậy đường chính KHÔNG phải fuzzy match nhãn, mà là **đọc thẳng cột mã số** rồi tra bảng tĩnh. Fuzzy/embedding/LLM chỉ là fallback khi mã số mất/mờ.

1. **Tầng 1 — Đọc mã số trực tiếp (chính, độ chính xác cao nhất):** lấy giá trị cột “Mã số” → tra TT200_CHART (dict tĩnh ~ vài trăm mã, seed từ phụ lục Thông tư 200). Hợp lệ ⇒ chấp nhận.
2. **Tầng 2 — Fuzzy match nhãn** (khi mã số thiếu/garbled): thư viện **RapidFuzz**, hàm `token_sort_ratio`, **ngưỡng** ≥ 88 . Tiền xử lý: lowercase, bỏ dấu câu, chuẩn hóa khoảng trắng. Khớp ⇒ chấp nhận.
3. **Tầng 3 — Embedding similarity** (khi fuzzy < 88): model `bkai-foundation-models/vietnamese-bi-enc` (hoặc `bge-m3`); **cosine**; encode nhãn vs embedding của toàn bộ nhãn chuẩn TT200; lấy top-1, **chấp nhận nếu** $\cos > 0.75$.
4. **Tầng 4 — LLM classify** (khi $\cos \leq 0.75$ hoặc top-2 sát nhau): đưa nhãn + top-5 ứng viên (từ tầng 3) cho LLM chọn, few-shot 3–5 ví dụ, *output rằng buộc* (chỉ 1 mã hoặc UNKNOWN).
5. **Fallback cuối:** không tầng nào đạt ⇒ gán `unmapped`, lưu nhãn thô, **đẩy HITL**. Tuyệt đối không bỏ âm thầm.

Prompt LLM (tầng 4):

He thống: Bạn là chuyên gia kế toán VN (TT200). Cho một tên chi tiêu và danh sách mã số ứng viên, chọn DÙNG MỘT mã phù hợp nhất, hoặc trả "UNKNOWN" nếu không chắc. Chỉ trả mã số.

Ví dụ:

- "Tiền và các khoản tương đương tiền" -> 110
- "Người mua trả tiền trước ngắn hạn" -> 312
- "Chi phí trả trước dài hạn" -> 261

Tên chi tiêu: "{label}"

Ứng viên: {top5_codes_with_names}

Trả lời:

Từ điển đồng nghĩa: seed thủ công từ phụ lục TT200; *tự lớn lên* — mỗi correction của người ở tầng fallback ghi thêm 1 cặp (nhãn thô → mã chuẩn) vào dictionary, lần sau khớp ngay ở tầng 2.

3.4 Validation số học (bắt lỗi extraction)

- $\text{Tong tai san (270)} == \text{Tai san NH (100)} + \text{Tai san DH (200)}$.
- $\text{Tong cong nguon von (440)} == \text{Tong tai san (270)}$.
- $\text{Cha} = \text{tổng các con theo cây phân cấp}$ (sai số cho phép do làm tròn ≤ 1 đơn vị).
- Vi phạm ⇒ cờ `extraction_warning` + đẩy HITL ô liên quan.

3.5 Schema chuẩn hóa (1 dòng, lưu Postgres)

```
{ "doc_id": "vnm_2023", "report": "balance_sheet", "nam": 2023,
  "ma_so": "110", "chi_tieu": "Tiền và tương đương tiền",
  "parent_ma_so": "100", "indent_level": 1, "thuyet_minh": "V.01",
  "gia_tri": 12345678901, "don_vi": "VND",
  "map_method": "direct_code", "map_confidence": 1.0,
  "provenance": { "page": 4, "bbox": [...] } }
```

4 Pha 3 — GraphRAG đa năm (chi tiết)

[QUYẾT ĐỊNH] Neo4j cho đồ thị; Microsoft GraphRAG (hoặc pipeline tự xây) cho indexing + community summary; embedding bge-m3 cho truy hồi ngữ nghĩa.

Mô hình đồ thị (Cypher rút gọn):

```
(:Company {mst, ten})
(:LineItem {ma_so, chi_tieu})
(:Value {gia_tri, nam, report})
(:Note {ma, noi_dung})

(LineItem)-[:HAS_VALUE]->(Value)
(Value)-[:OF_YEAR]->(:Year {nam})
(LineItem)-[:CHILD_OF]->(LineItem) //phan cap
(Value)-[:YOY {delta_pct}]->(Value) //nam-qua-nam
(LineItem)-[:REFERENCES]->(Note) //tham chieu thuyet minh
(LineItem)-[:COMPUTED_FROM]->(LineItem) //vi du LN gop = DT - GV
```

- Cạnh YOY tính sẵn lúc ingest $\Rightarrow$ truy vấn xu hướng nhanh.
- Community summary: gom cụm theo nhóm chỉ tiêu (thanh khoản, đòn bẩy...) để trả lời câu hỏi “toàn cục”.
- Chống bùng nổ node: chỉ mô hình hóa chỉ tiêu cốt lõi + thuyết minh quan trọng (whitelist mã số).

5 Pha 4 — Conformal Prediction (chi tiết)

Áp dụng cho bài toán phân loại ánh xạ nhãn $\rightarrow$ mã số TT200 (và tương tự cho cờ rủi ro).

[QUYẾT ĐỊNH] Split Conformal với APS (Adaptive Prediction Sets). Coverage target $1 - \alpha = 0.90$.

- **Điểm số:** bộ phân loại (embedding/LLM) cho phân phối $\hat{p}(y | x)$ trên các mã ứng viên.
- **Nonconformity score (APS):** với mẫu calibration (x, y) , sắp xếp $\hat{p}$ giảm dần, cộng dồn đến khi gặp lớp đúng $y \Rightarrow s =$ tổng tích lũy đó.
- **Calibration set:** 200–500 mẫu đã có nhãn người (golden + correction), KHÔNG dùng để train.
- **Ngưỡng:** $\hat{q}$ = phân vị $\lceil (n+1)(1 - \alpha) \rceil / n$ của tập điểm calibration.
- **Prediction set:** $C(x) = \{y : \text{APS score}(x, y) \leq \hat{q}\}$.
- **Luật abstain $\rightarrow$ HITL:**
 - $|C(x)| = 1 \Rightarrow$ tự động chấp nhận.
 - $|C(x)| > 1$ (mơ hồ) hoặc $|C(x)| = 0$ (lạ) $\Rightarrow$ **abstain**, đẩy người duyệt.
- **Khi có drift:** theo dõi độ phủ thực nghiệm trên cửa sổ trượt; nếu tụt dưới $1 - \alpha \Rightarrow$ recalibrate, hoặc dùng **Adaptive Conformal Inference (ACI)** cập nhật α_t online.
- **Thư viện:** MAPIE (hoặc tự cài ~ 30 dòng).

6 Pha 5 — LangGraph Agent (chi tiết)

6.1 Graph state schema

```
class AgentState(TypedDict):
    session_id: str
    task: str                # nhiệm vụ người dùng
    plan: list[str]          # bước đã phân ra
    cursor: int              # bước hiện tại
    messages: list           # hội thoại (LangChain msgs)
    retrieved: list[dict]    # số liệu lấy từ tool (có provenance)
    citations: list[dict]    # nguồn trích dẫn
    confidence: float
    needs_verification: bool
    answer: str | None
    abstained: bool
```

6.2 Các node & trách nhiệm

Node	Làm gì
planner	Phân rã task thành plan (chọn loại nhiệm vụ + tool cần dùng).
router	Điều phối: extract / compare / reconcile / qa; chọn tool kế tiếp theo cursor .
tool_executor	Gọi tool, ghi kết quả vào retrieved kèm provenance.
verifier	Nếu needs_verification : RAG quy định + web; tính confidence .
synthesizer	Viết answer + gán citations .
abstain	Trả “không đủ cơ sở” khi thiếu bằng chứng / confidence thấp.

6.3 Cạnh & điều kiện (conditional edges)

```
planner -> router
router -> tool_executor
tool_executor -> router          # nếu còn bước: cursor < len(plan)
tool_executor -> verifier        # nếu hết bước
verifier -> synthesizer          # if confidence >= 0.7 and citations != []
verifier -> tool_executor        # if thiếu dữ liệu (cần lấy thêm)
verifier -> abstain              # if confidence < 0.5 or no evidence
synthesizer -> END
abstain -> END
```

Hàm điều kiện ví dụ: `should_verify(state)` trả "verify" nếu task chạm quy định/cờ đỏ hoặc `confidence < 0.7`.

6.4 Tools (chữ ký + luật)

```
query_graph(intent|cypher) -> rows[+provenance]
get_lineitem(ma_so, nam) -> {gia_tri, provenance}
compute_ratio(name, nam) -> {value, formula, inputs[+provenance]}
reconcile(scope) -> [discrepancy{a,b,delta,severity}]
search_regulation(query) -> [chunk{text, cite}]
web_search(query) -> [result{title,url,snippet}]
```

Luật chống ảo giác (bắt buộc): agent CHỈ được lấy con số qua tool; cấm tự “nhớ”/sinh số. Mọi số trong **answer** phải truy được về **provenance** của một tool-call. Synthesizer từ chối câu trả lời chứa số không có nguồn.

6.5 Memory store

- **Ngắn hạn:** messages trong state; khi $> N$ token $\Rightarrow$ tóm tắt (summary node) giữ ngữ cảnh.
- **Dài hạn (theo session = 1 hồ sơ DN):**
 - Qdrant 1 collection, record {session_id, type:finding|decision|fact, content, embedding, citation, approved_by, ts}; truy hồi top-k lọc theo session_id.
 - Postgres bảng approved_findings cho recall chính xác + audit.
- **Checkpoint graph:** PostgresSaver (LangGraph) lưu state $\Rightarrow$ mở lại session, resume đúng chỗ.

7 Pha 6 — Vòng HITL & Finetune (chi tiết)

7.1 Định dạng dữ liệu correction (JSONL)

```
{ "doc_id":"vnm_2023","report":"balance_sheet","page":4,
  "table_id":"t1","cell":{"row":3,"col":2},
  "field":"gia_tri",           // gia_tri | ma_so | chi_tieu
  "predicted":"12.345.678.001","corrected":"12.345.678.901",
  "image_crop_path":"crops/vnm_2023_p4_t1_r3c2.png",
  "map_method_at_pred":"direct_code","confidence_at_pred":0.82,
  "corrected_by":"analyst_03","ts":"2026-06-19T10:00:00Z" }
```

7.2 Trigger huấn luyện lại (drift-based, không theo lịch)

Kích hoạt khi MỘT trong các điều kiện:

- Độ phủ conformal thực nghiệm (cửa sổ trượt) $< 1 - \alpha$ (vd < 0.90).
- PSI của phân phối confidence > 0.2 so với baseline.
- Số correction tích lũy ≥ 50 .

7.3 Quy trình finetune

1. Dựng tập train mới = synthetic gốc + toàn bộ correction (**oversample correction** $\times 3$).
2. Train **adapter LoRA MỚI** (adapter_vN+1), KHÔNG ghi đè adapter đang chạy. Cấu hình như Pha 2.
3. Đánh giá trên **golden eval set ĐỒNG BẰNG** (không đổi giữa các vòng).
4. Đăng ký adapter + metrics vào **MLflow**.

7.4 Chiến lược merge & phục vụ

[QUYẾT ĐỊNH] Không merge vào base mặc định. Phục vụ **base + active adapter** qua **vLLM** (hot-swap LoRA). Chỉ **merge_and_unload** khi ra bản ổn định để giảm overhead inference. Mỗi adapter versioned, “active” trở bằng con trở **active_adapter_id** trong DB/config.

7.5 Cổng promote & rollback

- **Promote** adapter mới $\rightarrow$ active CHỈ khi: accuracy golden $\geq$ active hiện tại $-\varepsilon$ ($\varepsilon = 0.5\%$) và không regression ở bất kỳ nhóm chỉ tiêu nào (per-category breakdown).
- **Shadow trước promote:** chạy adapter mới ở chế độ shadow trên mẫu, so sánh, rồi mới đổi con trở.
- **Rollback** = lật active_adapter_id về bản trước ($O(1)$, không tải lại base). Giữ tối thiểu 3 bản gần nhất.
- Ghi log + alert khi regression.

8 Pha 0/7/8/9 — Hạ tầng, hiệu năng, đánh giá, vận hành (chi tiết ngắn)

8.1 Pha 0 — Hạ tầng

docker-compose: postgres(+pgvector), redis, minio, qdrant, neo4j. Quản version dữ liệu/model bằng DVC + MLflow.

8.2 Pha 7 — Hiệu năng & song song

- **Celery + Redis:** 1 file = 1 task; concurrency = (số core -2). OCR trang song song trong file.
- **Idempotency:** key = SHA-256 file; đã xử lý $\Rightarrow$ bỏ qua.
- **Cache:** embedding cache (Redis); **semantic cache** câu hỏi lặp (so cosine $> 0.95 \rightarrow$ trả cache).
- **Pipeline streaming:** không chờ cả lô; file nào xong index ngay.
- **Serving:** vLLM batch; *fast-path* SLM rẻ, escalate model lớn khi confidence thấp.
- **Mục tiêu đo:** throughput (file/phút), p95 latency truy vấn.

8.3 Pha 8 — Đánh giá

Tầng	Metric	Mục tiêu
OCR	CER/WER (jiwer)	CER $< 3\%$ digital, $< 8\%$ scan
Bảng	TEDS	> 0.90
Trích xuất số	Cell accuracy	$\geq 95\%$
Ảnh xạ mã số	Mapping accuracy	$\geq 90\%$
Đối chiếu	Catch-rate (synthetic cấy lỗi)	$\geq 90\%$
Rủi ro	Backtest công ty đã vỡ/scandal	flag đúng ≥ 1 ca
Agent	% kết luận số có trích dẫn	100% (không bịa)

8.4 Pha 9 — Vận hành

Langfuse (trace agent/token-cost) + Prometheus/Grafana (hệ thống); audit log mọi kết luận; phân quyền duyệt; không dùng PII thật.

9 Phụ lục — Bảng tổng hợp ngưỡng & tham số

Tham số	Giá trị mặc định
Phát hiện digital (n_chars/trang)	≥ 50 & phủ $> 5\%$
Render scan	300 DPI
Fuzzy match (RapidFuzz token_sort)	≥ 88
Embedding accept (cosine)	> 0.75
Semantic cache hit (cosine)	> 0.95
LoRA (Qwen2-VL-2B)	r=16, alpha=32, dropout=0.05, lr=1e-4
LoRA (Donut decoder)	r=8..16
Conformal coverage $1 - \alpha$	0.90
Calibration set	200–500 mẫu có nhãn người
Agent verify khi	confidence < 0.7 hoặc chạm quy định
Agent abstain khi	confidence < 0.5 hoặc không có bằng chứng
Finetune trigger	coverage < 0.90 hoặc PSI > 0.2 hoặc ≥ 50 correction
Oversample correction	$\times 3$
Promote gate (ε)	accuracy không tụt quá 0.5%
Giữ adapter	≥ 3 bản gần nhất

— Hết bản spec chi tiết FinSight —